

Chapitre 2 : Vues statiques en UML

1. Introduction

UML offre un ensemble de diagrammes qui illustrent les aspects quantifiables d'un système, et qui peuvent être décrits visuellement, tels que les relations, le comportement, la structure ou la fonctionnalité. Ces diagrammes peuvent être classés en deux catégories de vues : Les *vues statiques* et les *vues dynamiques*.

Pour les vues statiques, ils regroupent l'ensemble de diagrammes statiques ou ce qu'on appelle les diagrammes de structure. Ils montrent les aspects statiques ou structurels d'un système et décrivent les composants qui constituent ce système et les relations entre eux.

Les diagrammes statiques les plus importants rassemblent :

- Le diagramme de classes
- Le diagramme d'objets
- Le diagramme de composants

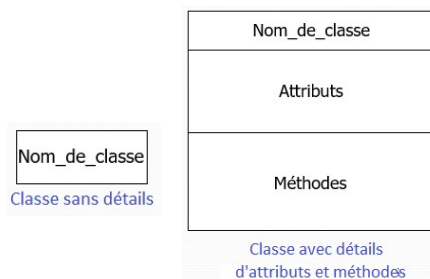
2. Diagramme de classes

Nous utilisons les diagrammes de classes pour décrire la structure du système. Les classes sont des abstractions qui spécifient la structure et le comportement commun d'un ensemble d'objets.

Les diagrammes de classes décrivent le système en termes d'objets, de classes, d'attributs, d'opérations et de leurs associations. Un diagramme de classe est utilisé pour représenter la structure d'un système en termes d'objets, de leurs attributs et leurs relations. Un diagramme de classes est donc une collection d'éléments de modélisation statiques qui montre la structure d'un modèle. Il fait abstraction des aspects dynamiques et temporels.

2.1 Représentation graphique

La représentation graphique d'une classe est comme suit :



Exemple 1 : Classes sans détails

Elaboration d'un diagramme de classes décrivant les éléments de toutes les montres de la classe SimpleMontre. Ces objets montres ont tous une association vers un objet de la classe AppuiBouton, un objet de la classe Afficheur, un objet de la classe Heure, et un objet de la classe Batterie.

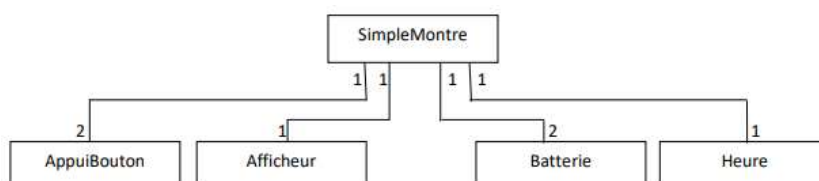
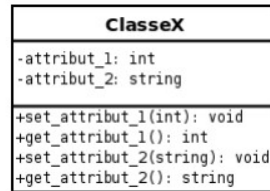


Diagramme de classe UML représentant les éléments de Simple montre

Les numéros sur les extrémités des associations indiquent le nombre de liens que chaque objet SimpleMontre peut avoir avec un objet d'une classe donnée. Par exemple, une SimpleMontre a exactement deux boutons, un afficheur, deux batteries, et une heure. De même, tous les objets AppuiButton, Afficheur, Heure, et Batterie sont associés à un seul objet SimpleMontre.

Exemple 2 : Classe avec détails



La notion de visibilité indique qui peut avoir accès à l'attribut.

Elle ne peut prendre que quatre valeurs :

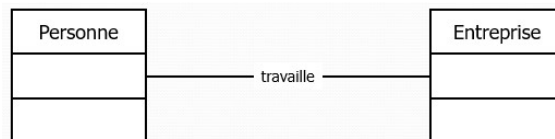
Caractère	Rôle	Mot clé	Description
+	accès public	public	Toutes les autres classes ont accès à cet attribut.
#	accès protégé	protected	Seules la classe elle-même et les classes filles (héritage) ont accès à cet attribut.
~	accès package	package	Classe visible uniquement dans le package.
-	accès privé	private	Seule la classe elle-même a accès à cet attribut.

Afin de respecter le principe fondamental d'**encapsulation**, tous les attributs devraient être *privés*.

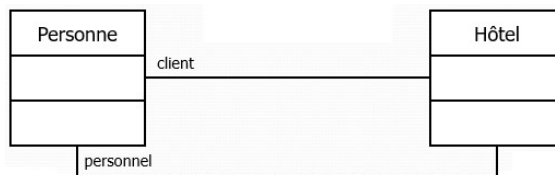
2.2 Relations entre classes

A. Association :

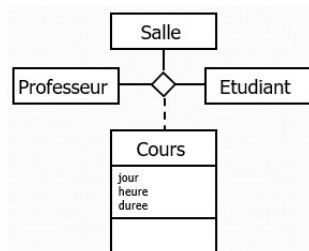
Une relation ou association entre classes décrit un ensemble de liens (instances de l'association). Les cardinalités (ou multiplicité) indiquent le nombre d'instances d'une classe pour chaque instance de l'autre classe. Généralement, un verbe donne le nom de l'association.



Une association peut définir un rôle pour les participants :



Une association reliant plus de deux classes est une association n-à-n :



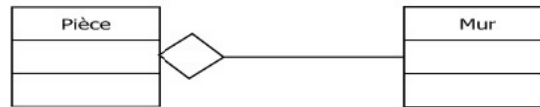
B. Agrégation :

Une agrégation est une association qui représente une relation d'inclusion structurelle ou comportementale d'un élément dans un ensemble.

Une agrégation peut notamment (mais pas nécessairement) exprimer :

- qu'une classe (un "élément") fait partie d'une autre ("l'ensemble"),
- qu'un changement d'état d'une classe, entraîne un changement d'état d'une autre,
- qu'une action sur une classe, entraîne une action sur une autre.

Exemple :

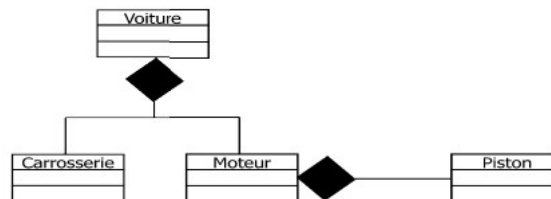


C. Composition :

La composition, également appelée agrégation composite, décrit une contenance structurelle entre instances. Ainsi, la destruction de l'objet composite implique la destruction de ses composants.

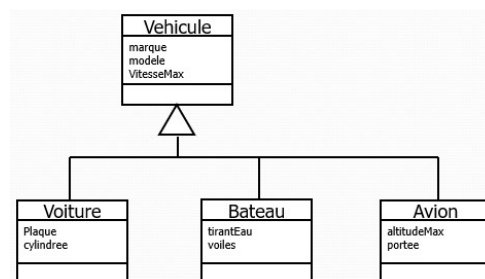
- Cas particulier d'agrégation : contenance physique
- Représente une relation de type "composé / composant"
- Les cycles de vies des composants et du composé sont liés : si le composé est détruit (ou copié), ses composants le sont aussi
- A un même moment, une instance de composant ne peut être liée qu'à un seul composé

Exemple :



D. Généralisation et Héritage :

La généralisation décrit une relation entre une classe générale (classe de base ou classe parent) et une classe spécialisée (sous-classe). La classe spécialisée est intégralement cohérente avec la classe de base, mais comporte des informations supplémentaires (attributs, opérations, associations). Une relation d'héritage est schématisée par une ligne terminée par un triangle. Le triangle pointe vers la super-classe, et dont l'autre extrémité est fixée à la sous-classe.



Quand une généralisation ne sert qu'à des fins de modélisation d'attributs et d'opérations partagés, c'est à dire si la généralisation n'est jamais instanciée, elle est appelée **classe abstraite**.

3. Diagramme d'objets

Pour représenter un contexte précis, un diagramme de classes peut être instancié en diagrammes d'objets. Un diagramme d'objets représente un ensemble d'objets et leurs liens. C'est une instance d'un diagramme de classes.

Les objets sont donc des instances de classes qui sont créées, modifiées, et détruites lors de l'exécution du système. Les objets ont un état qui comprend les valeurs de leurs attributs et leurs relations avec d'autres objets. Un objet a une identité et stocke les valeurs d'attributs. Chaque objet appartient exactement à une classe.

Les diagrammes d'objets sont des vues statiques des instances des éléments qui apparaissent dans les diagrammes de classes. Ils présentent la vue de conception d'un système, exactement comme les diagrammes de classes, mais à partir de cas réel ou de prototypes.

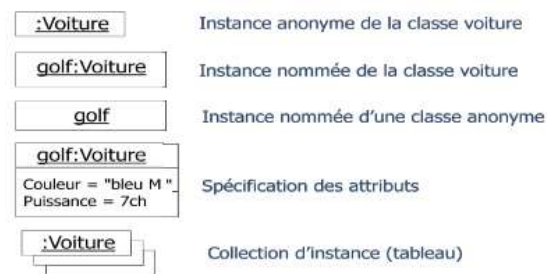
Graphiquement, un objet se représente comme une classe. Cependant, le compartiment des opérations n'est pas utile. De plus, le nom de la classe dont l'objet est une instance est précédé d'un << : >> et est souligné. Pour différencier les objets d'une même classe, leur identifiant peut être ajouté devant le nom de la classe. Enfin les attributs reçoivent des valeurs.

Quand certaines valeurs d'attribut d'un objet ne sont pas renseignées, on dit que l'objet est partiellement défini. Dans un diagramme d'objets, les relations du diagramme de classes deviennent des liens. La relation de généralisation ne possède pas d'instance, elle n'est donc jamais représentée dans un diagramme d'objets.

Dans le monde réel, de nombreux objets sont liés entre eux. Ce lien correspond à une association qui existe entre les objets. En UML, ces liens sont décrits par une association, comme un objet est décrit par une classe. Un lien est une occurrence d'une association. Par conséquent, une association relie des classes. Chaque occurrence de cette association relie entre elles des instances de ces classes.

Une association porte un nom. Comme pour une classe, ce nom est significatif des occurrences de l'association.

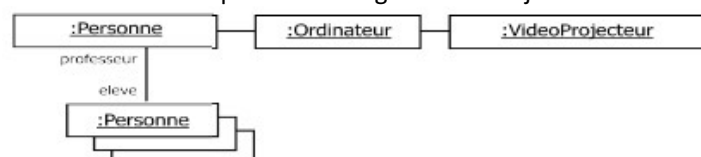
Graphiquement, un lien se représente comme une relation, mais, s'il y a un nom, il est souligné. Naturellement, on ne représente pas les multiplicités.



Exemple d'objets



Exemple 1 d'un Diagramme d'objets



Exemple 2 d'un Diagramme d'objets

3. Diagrammes de Composants

3.1 Introduction

Les Diagrammes de Composants UML sont des outils visuels qui permettent de représenter la structure et l'organisation des composants logiciels et matériels au sein d'un système. Ils offrent une vue détaillée des interactions entre ces composants, facilitant ainsi la compréhension et la communication dans le processus de développement logiciel. Ils complètent d'autres diagrammes UML tels que les Diagrammes de Classes, de Séquence et de Déploiement pour fournir une vue globale du système tout au long du cycle de vie du développement logiciel.

Les Diagrammes de Composants permettent aux équipes de développement de mieux comprendre la composition d'un système et la manière dont ses parties interagissent et mettent en évidence les dépendances entre les composants, facilitant ainsi la gestion des relations complexes au sein d'un système.

3.2 Concept de composant

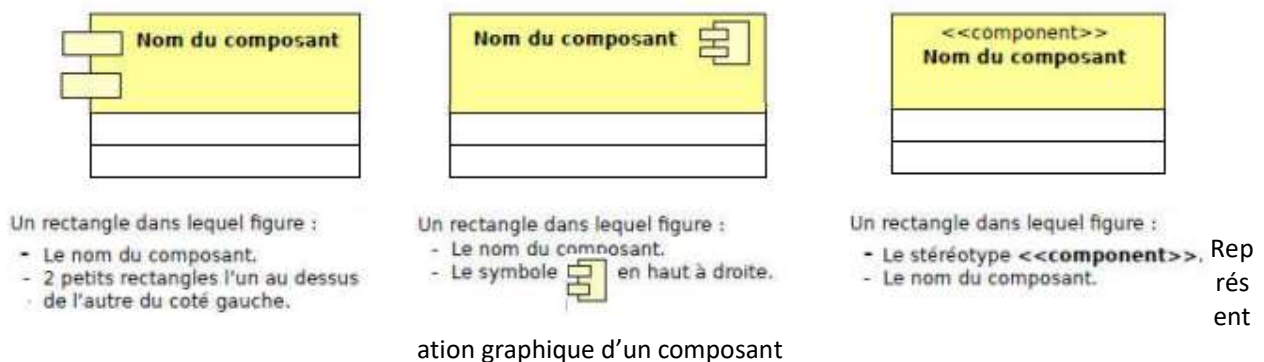
Dans la modélisation UML, les composants sont des éléments de modèle qui représentent des éléments indépendants, interchangeables d'un système. Ils respectent et réalisent une ou plusieurs interfaces fournies et obligatoires, qui déterminent le comportement des composants.

Les composants représentent des unités modulaires et autonomes du système. Un composant peut être une classe, un module, un service ou même un matériel physique. La modélisation des composants permet de décomposer un système complexe en éléments plus gérables, facilitant ainsi le développement, la maintenance et la compréhension du système. Les composants rendent un système plus souple, plus évolutif et plus réutilisable.

Les composants logiciels sont des entités autonomes qui encapsulent des fonctionnalités spécifiques. Dans un Diagramme de Composants UML, ils sont représentés graphiquement avec des boîtes, illustrant ainsi leur rôle dans le système global. En complément des composants logiciels, les Diagrammes de Composants peuvent également inclure des composants matériels. Ces derniers représentent le matériel physique nécessaire au fonctionnement du système, tels que des serveurs, des ordinateurs, ou d'autres dispositifs. L'intégration de composants matériels offre une vision complète de l'architecture système, combinant les aspects logiciels et matériels.

A. Représentation graphique d'un composant :

Un composant prend habituellement le nom de l'élément du système qu'il représente. Il existe différentes façons pour représenter un composant :



Les compartiments sont utilisés pour afficher des informations sur les attributs, les opérations, les interfaces fournies, les interfaces obligatoires, les réalisations et la structure interne du composant.

B. Les interfaces du composant :

Le comportement interne d'un composant, généralement réalisé par un ensemble de classes, est totalement masqué : seules ses interfaces sont visibles. La seule contrainte pour pouvoir substituer un composant par un autre est de respecter les interfaces requises et offertes. Les plugins, les bibliothèques sont des exemples de composants.

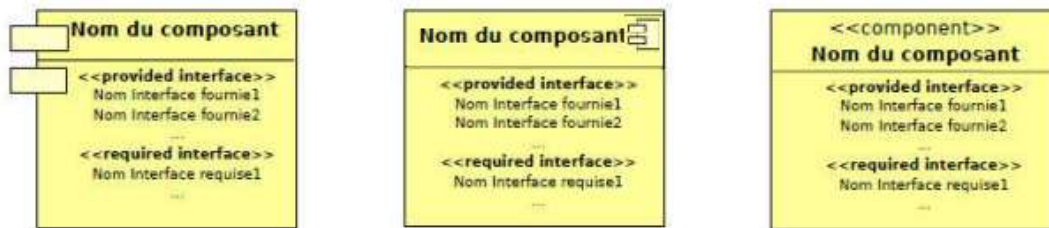
Les interfaces définissent les contrats entre les composants, spécifiant les méthodes, opérations ou services que chaque composant expose ou requiert. Elles facilitent la communication entre les composants en déterminant comment ils interagissent les uns avec les autres.

Il existe deux types d'interfaces :

- **Les interfaces requises** : Ce sont des interfaces qui fournissent un service au composant et dont il a besoin pour fonctionner.
- **Les interfaces fournies** : Ce sont des interfaces par lesquels le composant fournit lui-même un service.

Il existe plusieurs possibilités pour représenter les interfaces :

1. Intégrées dans la représentation du composant en utilisant les stéréotypes <<required interface>> et <<provided interface>>.



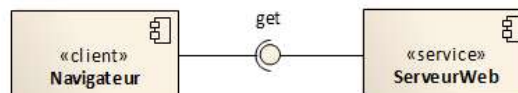
2. Dans un classeur séparé du composant dans lequel sont listés les différents services.



3. Avec des connecteurs d'assemblage :

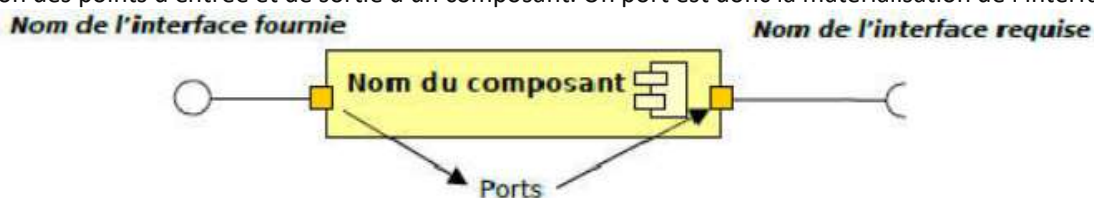


Un assemblage entre deux composants est lorsqu'une même interface est requise pour un composant et fournie par l'autre :



C. Les ports du composant :

Les ports sont des points d'interaction spécifiques sur un composant, permettant la connexion avec d'autres composants via des connecteurs. Les ports jouent un rôle crucial dans la modélisation des échanges d'information et la définition des points d'entrée et de sortie d'un composant. Un port est donc la matérialisation de l'interface.

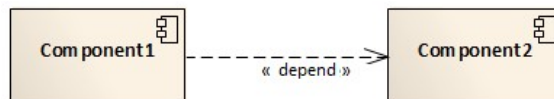


D. Associations entre composants :

Les associations entre composants (connecteurs) représentent des connexions significatives entre les différentes unités modulaires du système et offrent une représentation visuelle des liens fonctionnels ou structurels qui existent entre les composants.

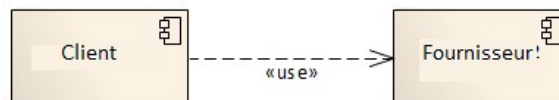
Dans un Diagramme de Composants, les associations sont généralement représentées par des lignes reliant les composants concernés. La direction de la flèche indique la nature de la relation, soulignant la dépendance entre les composants. Une flèche pointant d'un composant A vers un composant B signifie souvent que A dépend de B d'une manière ou d'une autre. Il existe plusieurs types d'Associations :

- **Dépendance** : Indique qu'un composant dépend d'un autre, ce qui signifie que des modifications dans le composant dépendant peuvent affecter le composant dépendu.



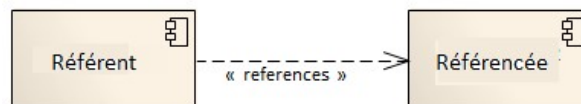
Dans cet exemple, le composant 1 dépend du composant 2, et cela peut signifier que des modifications dans le composant 2 peuvent avoir un impact sur le composant 1.

- **Usage** : Elle exprime une dépendance fonctionnelle où un composant, appelé le client, dépend d'un autre composant, appelé le fournisseur, pour accéder à certaines fonctionnalités.



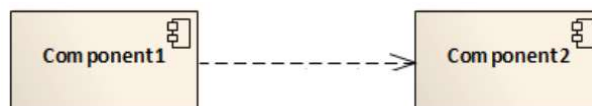
Dans cet exemple, le composant de gestion des utilisateurs (Client) peut utiliser les services d'un composant de stockage de données (fournisseur) pour accéder et manipuler les données des utilisateurs.

- **Référence** : Elle exprime une relation où un composant (appelé référent) fait référence à un autre composant (appelé référencé) pour utiliser ses spécifications, mais sans nécessairement impliquer une dépendance fonctionnelle directe.



Dans cet exemple, le composant graphique (Référent) peut référencer une bibliothèque de gestion d'images (Référencé) pour utiliser des fonctionnalités de traitement d'images sans être directement dépendant du fonctionnement interne de la bibliothèque.

- **Association Simple** : Représente une connexion plus générique entre deux composants sans spécifier une dépendance particulière. Une flèche indique la direction de l'association, mais la nature précise de la relation (utilisation, dépendance, etc.) n'est pas spécifiée.



Dans cet exemple, il y a une relation générique entre le composant 1 et le composant 2.

Comprendre et modéliser correctement les associations entre composants est essentiel pour garantir une conception logicielle cohérente.

3. Relation entre diagramme de composants et diagramme de classes

La relation entre les Diagrammes de Composants et les Diagrammes de Classes réside dans le fait qu'un Diagramme de Composants utilise souvent des classes pour représenter les composants. Chaque composant peut encapsuler une ou plusieurs classes, et les interfaces définies dans les classes peuvent être exposées via les ports des composants dans les Diagrammes de Composants. Ainsi, les classes identifiées dans un Diagramme de Classes peuvent être utilisées pour spécifier le contenu des composants dans un Diagramme de Composants. Les deux diagrammes contribuent donc à une modélisation complète d'un système logiciel.